

VHDL Design and Validation of Serial communication Bus for FPGA

Dr. T. Nandha Kumar

Senior Member IEEE, C.Eng.(UK), Fellow HEA (UK)

Associate Professor

Department of Electrical and Electronic Engineering

The University of Nottingham

Malaysia

Email: nandhakumaar.t@nottingham.edu.my

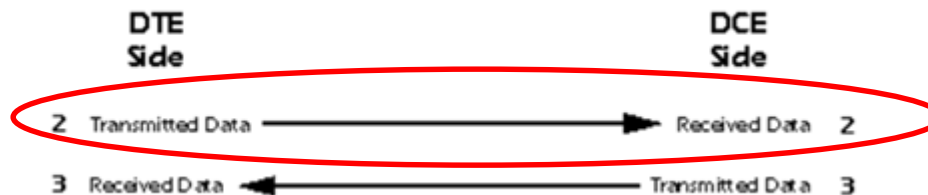
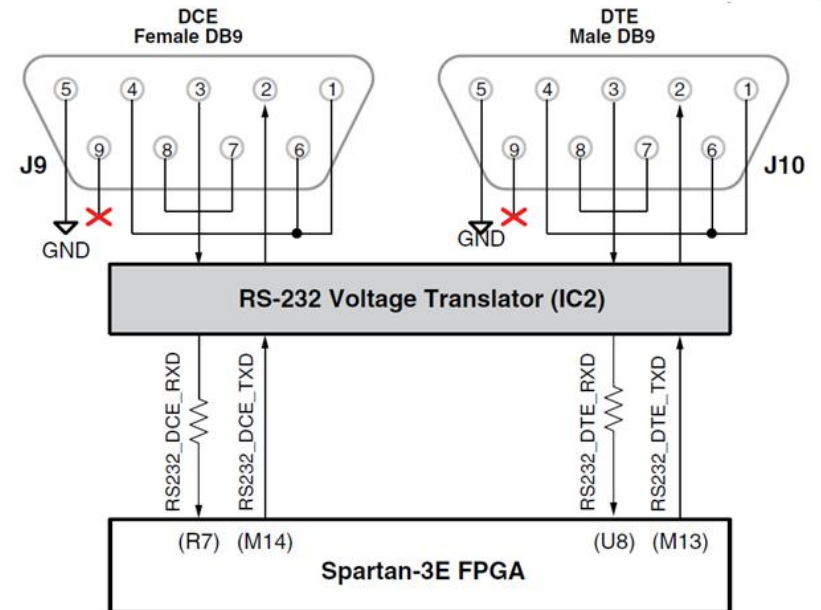
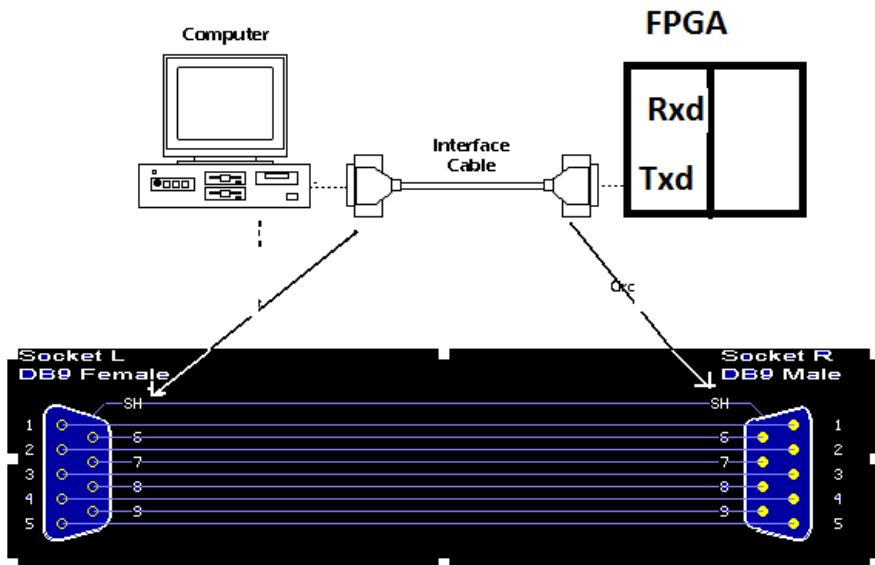
Objectives

- VHDL Digital Design of serial communication bus RS232 & SPI (Specification, Design, Validation)
- VHDL Testbench for RS232
- VHDL constructs for Testbench Design
- Validation of Rx block of RS232 using simulation

Serial Bus

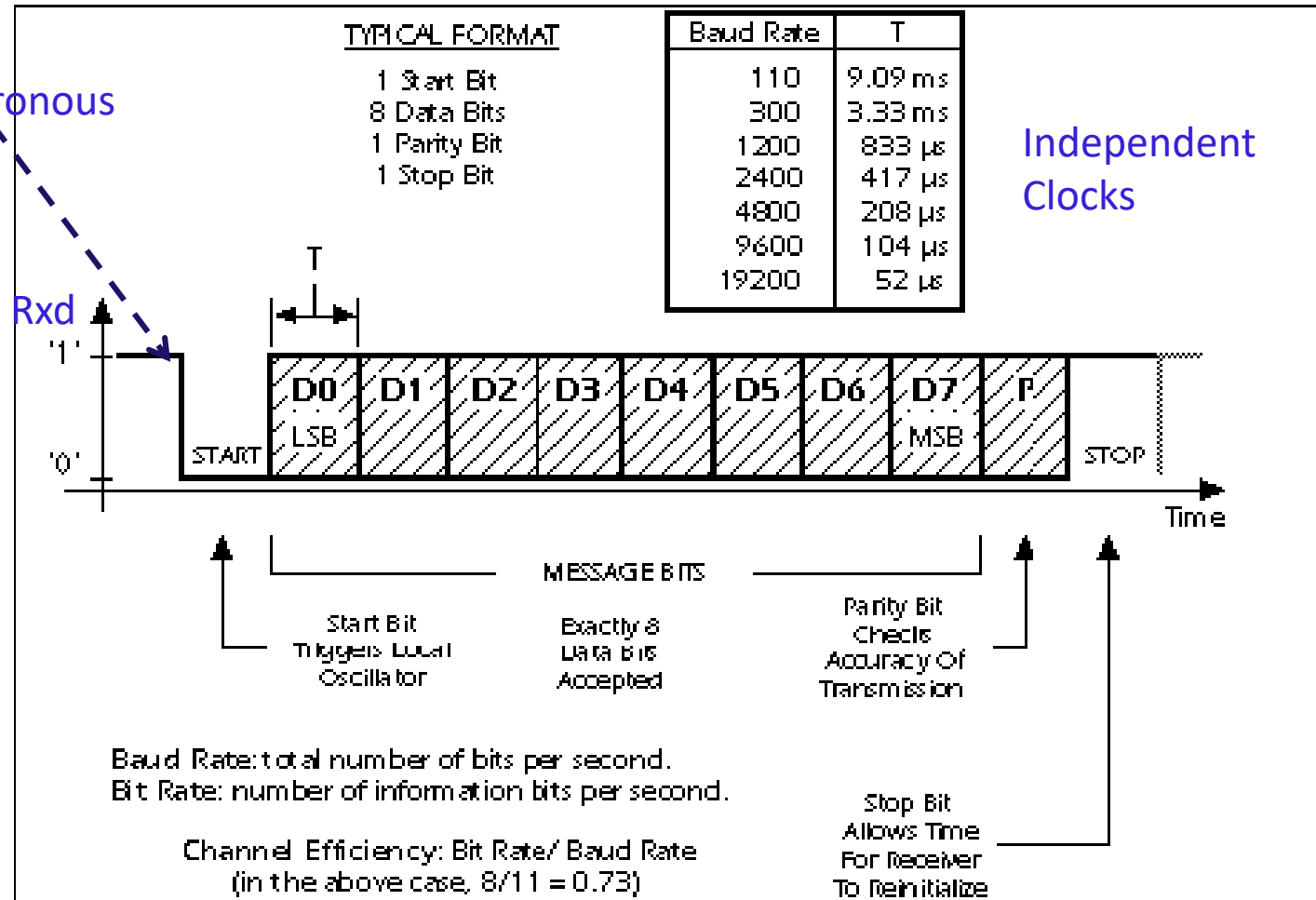
- I2C
- RS232 or UART
- SPI
- USB
- Etc....
- Used in wireless modules, Bluetooth modules, modems etc...

RS232



RS232 Protocol

Asynchronous

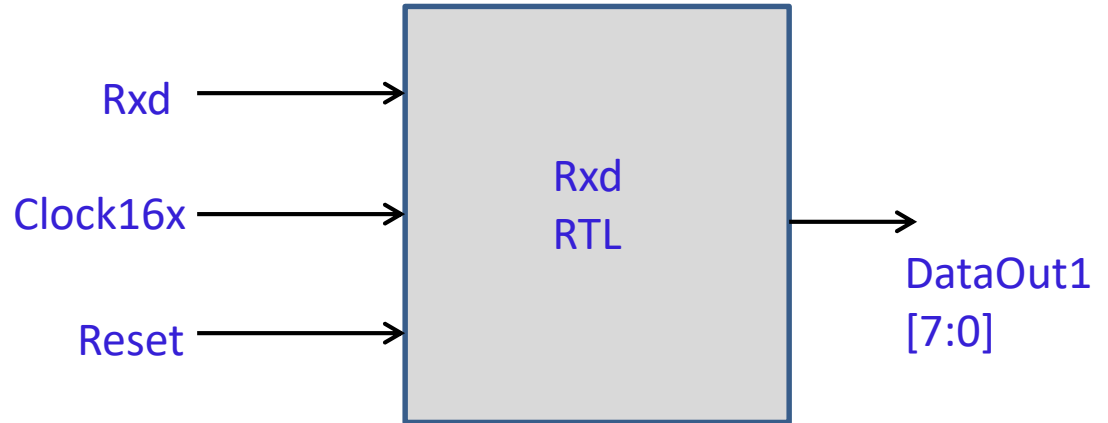


$$\text{Effective data rate (bytes/sec)} = (\text{Baud rate} * \text{Channel efficiency})/8 = 960\text{bytes/sec}$$

Rxd Design Spec

- Rxd
 - Asynchronous Signal, High at Idle
 - High to Low – Start bit
 - Low to High – Stop bit
 - Receive Data at a frequency set by Txd;
Synchronize Txd and Rxd clocks (Data Clock).
 - Data Transmission LSB to MSB of 8 bit length

Rxd Top level



```
entity RS232Rxd is
    port( Reset, Clock16x, Rxd: in std_logic;
          DataOut1: out std_logic_vector (7 downto 0));
end RS232Rxd;
```

Rxd VHDL

- Rxd
- How to handle Asynchronous Signal?
- How to detect START bit?

```
process (Clock16x)
begin
```

Rxd VHDL

```
    if rising_edge (Clock16x) then
        if Reset = '1' or iReset = '1' then
            iRxd1 <= '1';
            iRxd2 <= '1';
            iClock1xEnable <= '0';
            iClockDiv <= (others=>'0');
        else
            iRxd1 <= Rxd;
            iRxd2 <= iRxd1;
        end if;
        --detect Rxd falling edge
        if iRxd1 = '0' and iRxd2 = '1' then
            iClock1xEnable <= '1';
        end if;
        --Gated clock
        if iClock1xEnable = '1' then
            iClockDiv <= iClockDiv + '1';
        end if;
    end if;
end process;
-- Generation of Rxd clock
iClock1x <= iClockDiv(3);
```

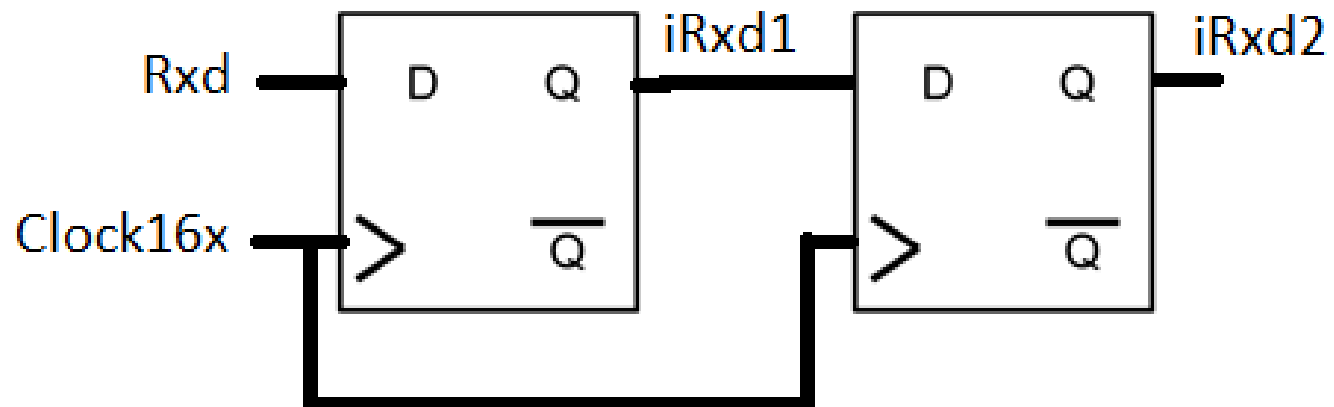
Sampling Frequency
16x Baud rate

Double Sync Flip-flop

Rxd VHDL

Double Sync Flip-flop

```
iRxd1 <= Rxd;  
iRxd2 <= iRxd1;
```



```
process (Clock16x)
begin
```

Rxd VHDL

```
    if rising_edge (Clock16x) then
        if Reset = '1' or iReset = '1' then
            iRxd1 <= '1';
            iRxd2 <= '1';
            iClock1xEnable <= '0';
            iClockDiv <= (others=>'0');
        else
            iRxd1 <= Rxd;
            iRxd2 <= iRxd1;
        end if;
        --detect Rxd falling edge
        if iRxd1 = '0' and iRxd2 = '1' then
            iClock1xEnable <= '1';
        end if;
        --Gated clock
        if iClock1xEnable = '1' then
            iClockDiv <= iClockDiv + '1';
        end if;
    end if;
end process;
-- Generation of Rxd clock
iClock1x <= iClockDiv(3);
```

Sampling Frequency
16x Baud rate

Double Sync Flip-flop

Detect Falling edge

Gated Clock + Counter

Data clock; Divide by 16

Rxd VHDL

- How to accumulate the received serial data?
- How to know the STOP bit?
- When to send the accumulated data?
- How to Restart the process?

Rxd VHDL

```
process (iClock1xEnable, iClock1x)
begin
    if iClock1xEnable = '0' then
        iNoBitsReceived <= (others=>'0');
        presState <= stIdle;
    elsif rising_edge (iClock1x ) then
        iNoBitsReceived <= iNoBitsReceived + '1';
        presState <= nextState;
    end if;

    if rising_edge (iClock1x) then
        if iEnableDataOut = '1' then
            iDataOut1 <= iShiftRegister;
        else
            iShiftRegister <= Rxd & iShiftRegister(7 downto 1);
        end if;
    end if;
end process;
```

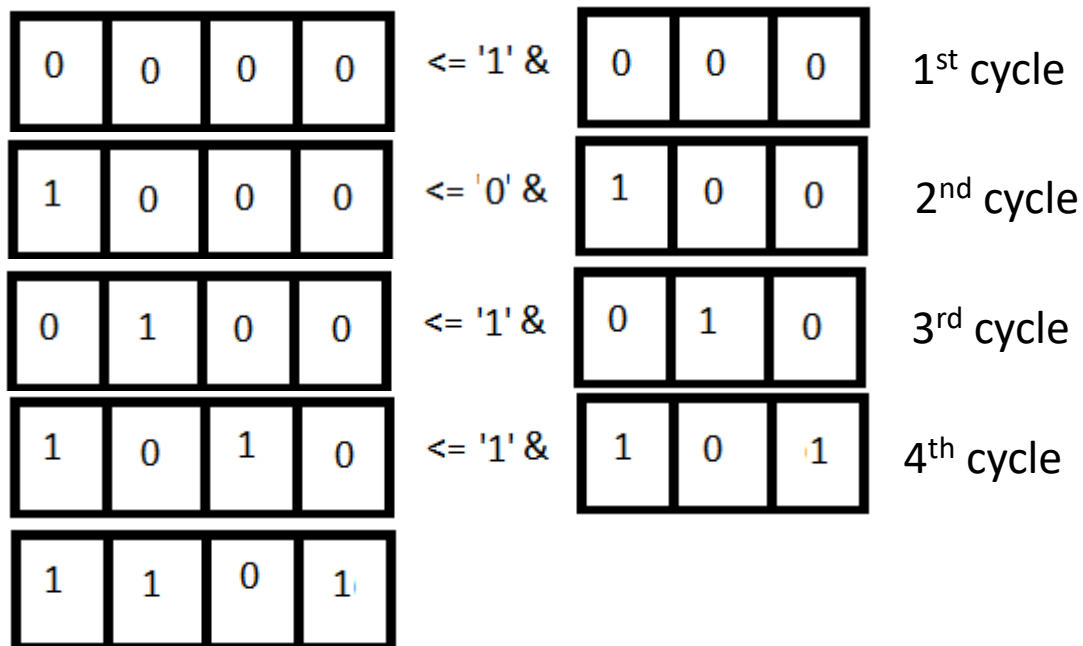


Accumulate serial data

```
DataOut1 <= iDataOut1;
```

Rxd VHDL

- Accumulate Serial Data
 - Assume ishiftregister [3:0] & Serial data “1101”
 - iShiftRegister <= Rxd & iShiftRegister(7 downto 1);



Rxd VHDL

```
process (iClock1xEnable, iClock1x)
begin
```

```
    if iClock1xEnable = '0' then
        iNoBitsReceived <= (others=>'0');
        presState <= stIdle;
    elsif rising_edge (iClock1x) then
        iNoBitsReceived <= iNoBitsReceived + '1';
        presState <= nextState;
    end if;
```

Help in STOP bit

```
    if rising_edge (iClock1x) then
        if iEnableDataOut = '1' then
            iDataOut1 <= iShiftRegister;
        else
            iShiftRegister <= Rxd & iShiftRegister(7 downto 1);
        end if;
    end if;
```

Send out the data

```
end process;
```

Accumulate serial data

```
DataOut1 <= iDataOut1;
```

Rxd VHDL

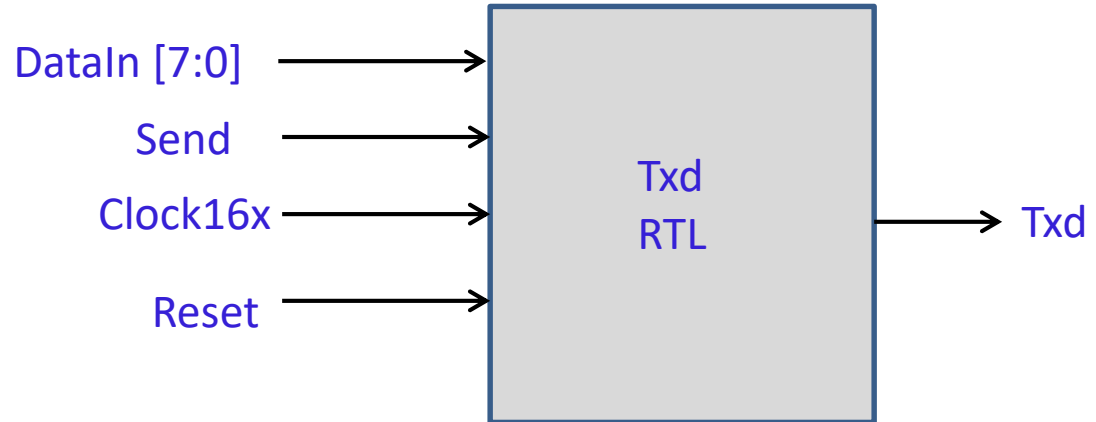
```
process (presState, iClock1xEnable, iNoBitsReceived)
begin
-- signal defaults
iReset <= '0';
iEnableDataOut <= '0';
case presState is
when stIdle =>
    if iClock1xEnable = '1' then
        nextState <= stData;
    else
        nextState <= stIdle;
    end if;
when stData =>
    if iNoBitsReceived = "1001" then
        iEnableDataOut <= '1';
        nextState <= stStop;
    else
        iEnableDataOut <= '0';
        nextState <= stData;
    end if;
when stStop =>
    nextState <= stRxdCompleted;
when stRxdCompleted =>
    iReset <= '1';
    nextState <= stIdle;
end case;
end process;
```

Send out the data

Help in STOP bit

Reset the process

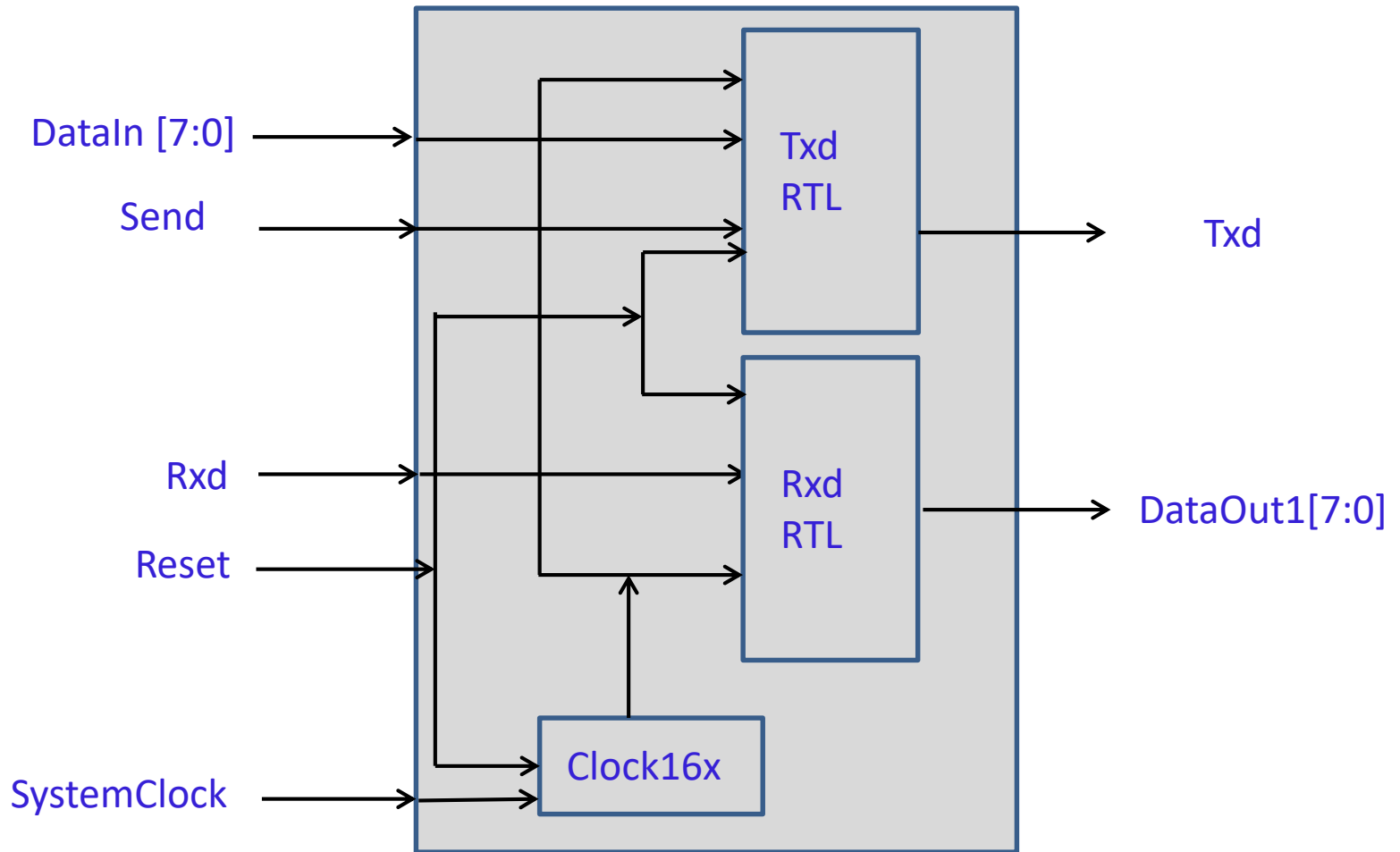
Txd Top level



```
entity RS232Txd is
port (
    Reset, Clock16x: in std_logic;
    Send: in std_logic := '0';
    DataIn: in std_logic_vector(7 downto 0);
    Txd: out std_logic );
end RS232Txd;
```

Try your Design !!!

RS232 Top Level

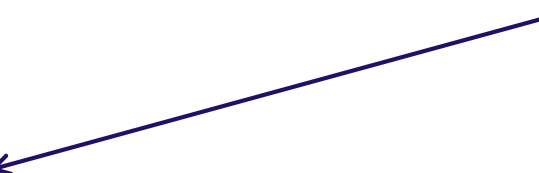


RS232 Top Level VHDL

```
entity RS232_Toplevel is
    port( Reset, SystemClock, Rxd, Send: in std_logic;
          DataIn: in std_logic_vector(7 downto 0);
          DataOut: out std_logic_vector (7 downto 0);
          Txd: out std_logic);
end RS232_Toplevel;

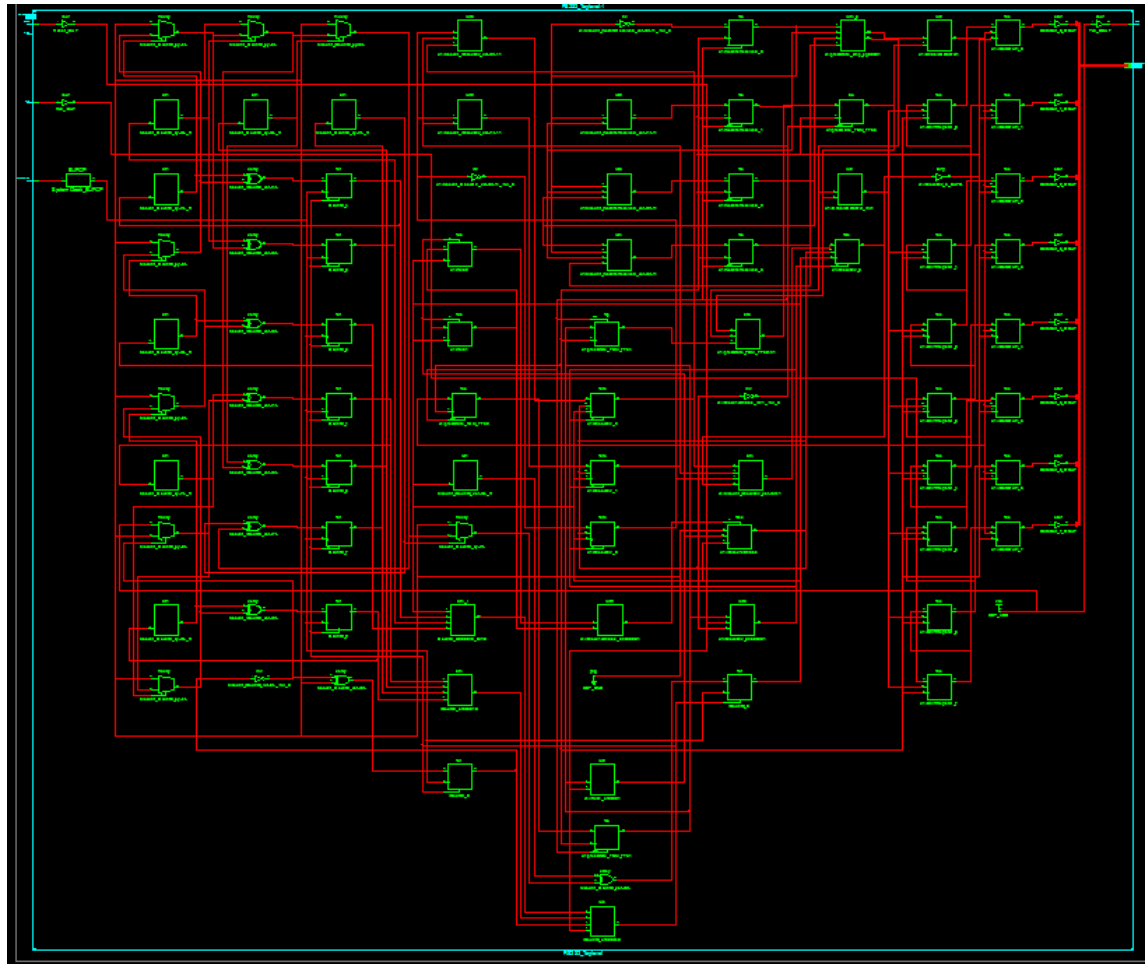
process (SystemClock)
begin
    if rising_edge(SystemClock) then
        if Reset = '1' then
            iCount9 <= (others=>'0');
        elsif
            iCount9 = "101000101" then -- the divider is 325, or "101000101"
                iCount9 <= (others=>'0');
            else iCount9 <= iCount9 + '1';
            end if;
        end if;
    end if;
end process;

iClock16x <= iCount9(8);
```

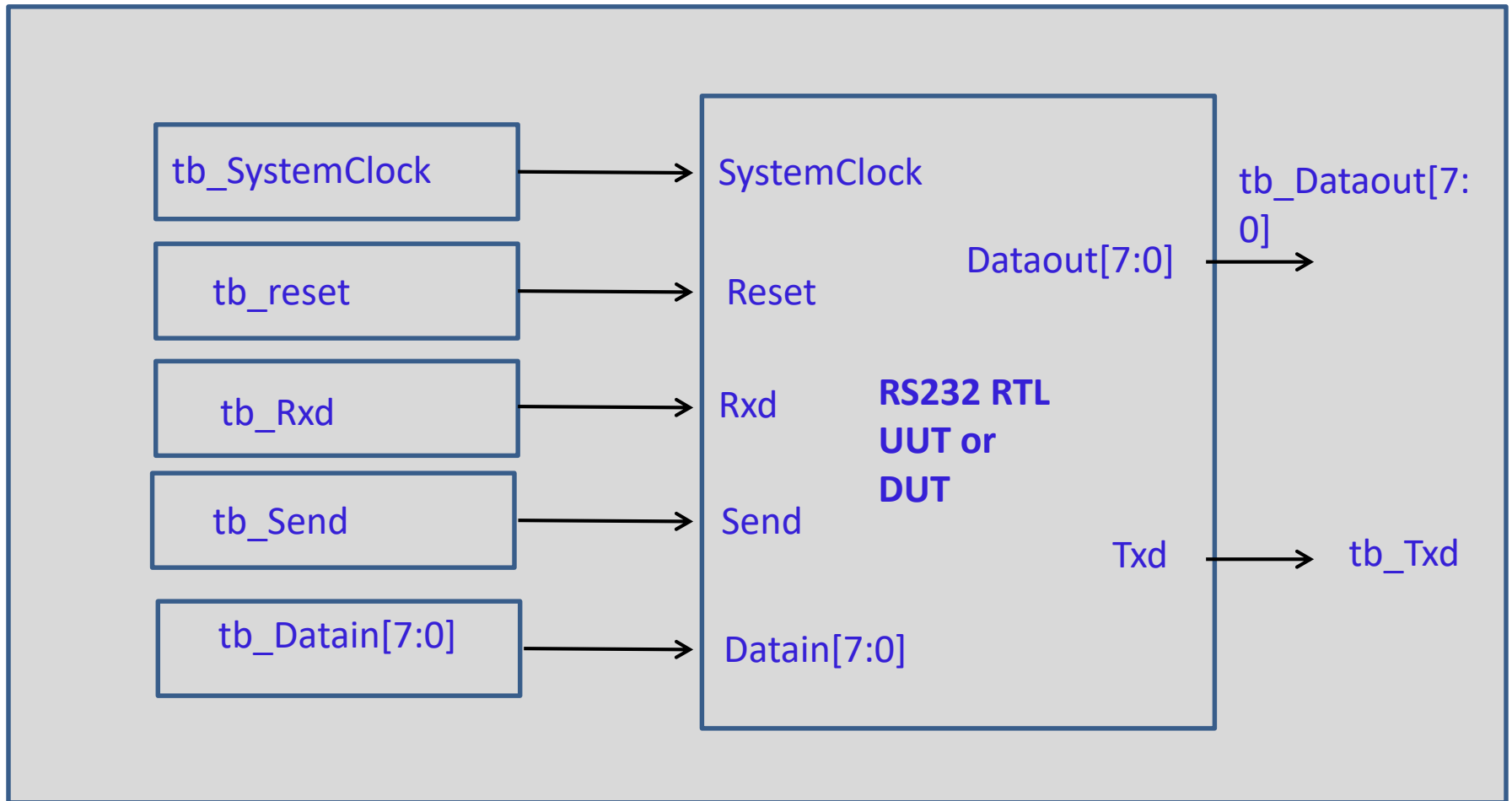


Clock16X Generation

FPGA Implementation



RS232 VHDL Testbench



VHDL Testbench –RS232

- System Clock 50MHz

```
constant tb_SystemClock_period : time := 20ns;
constant tb_Clock16x_period : time := 6.5us;

BEGIN

    -- System Clock Generation
    process
    begin
        tb_SystemClock <= '0';
        wait for tb_SystemClock_period/2;
        tb_SystemClock <= '1';
        wait for tb_SystemClock_period/2;
    end process;
```

VHDL Testbench –RS232

- Reset

```
-- Rxd Stimulus
reset: process
begin
    -- hold reset state for 100 ns.
    tb_Reset <= '1';
    wait for 1.5*tb_Clock16x_period;
    tb_reset <= '0';
    wait;
end process;
```

VHDL Testbench –RS232

- Rxd Serial data

```
Rxd: process
begin
    tb_rxd <= '1';
    wait for 5.5*tb_Clock16x_period;
    tb_rxd <= '0'; -- Start bit
    wait for 16*tb_Clock16x_period;
    tb_rxd <= '0'; -- Bit 0
    wait for 16*tb_Clock16x_period;
    tb_rxd <= '1'; -- Bit 1
    wait for 16*tb_Clock16x_period;
    tb_rxd <= '0'; -- Bit 2
    wait for 16*tb_Clock16x_period;
    tb_rxd <= '0'; -- Bit 3
    wait for 16*tb_Clock16x_period;
    tb_rxd <= '0'; -- Bit 4
    wait for 16*tb_Clock16x_period;
    tb_rxd <= '1'; -- Bit 5
    wait for 16*tb_Clock16x_period;
    tb_rxd <= '1'; -- Bit 6
    wait for 16*tb_Clock16x_period;
    tb_rxd <= '0'; -- Bit 7
    wait for 16*tb_Clock16x_period;
    tb_rxd <= '1'; -- Stop bit
    wait for 16*tb_Clock16x_period;
    wait;
```

```
end process;
```

VHDL Testbench –RS232

- Txd data

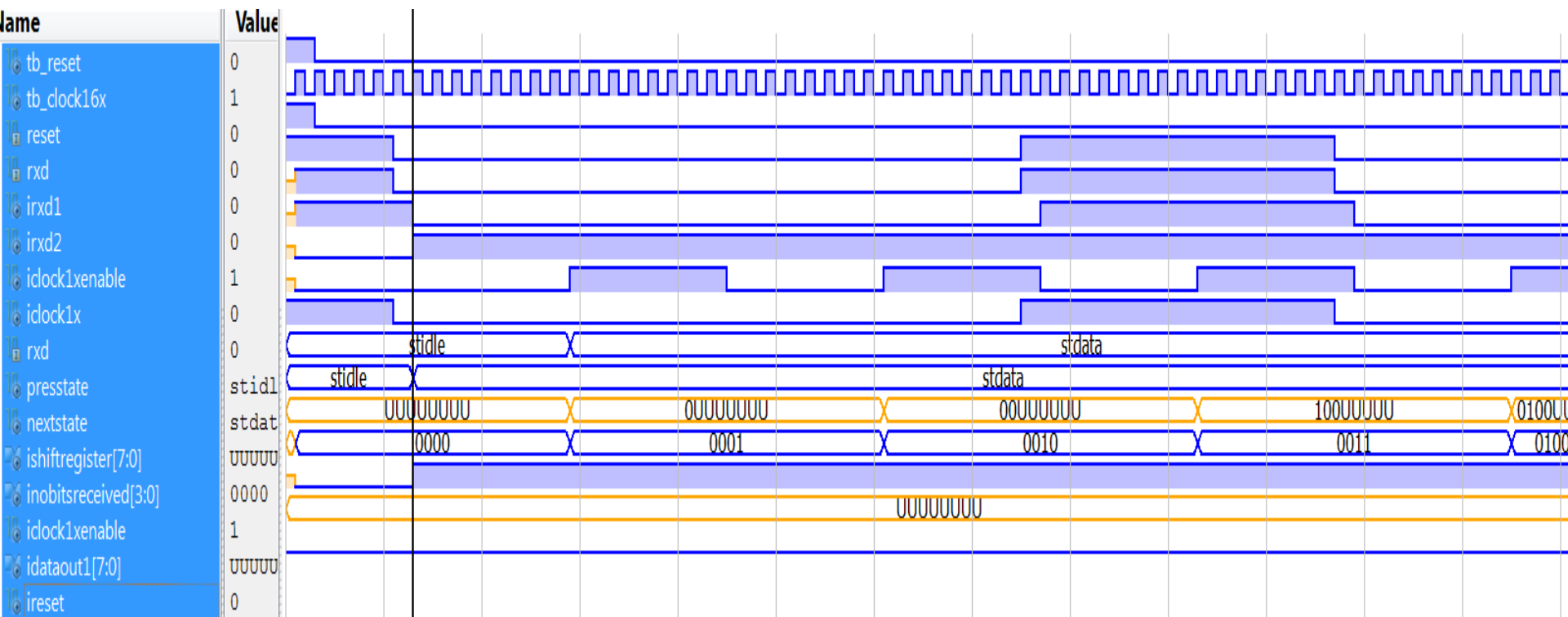
```
-- Txd Stimulus
-- produce Send signal
process
begin

    tb_send <= '0';
    wait for 200*tb_Clock16x_period;
    tb_send <= '1';
    wait for tb_Clock16x_period;
    wait;
end process;

-- Txd Data
process
begin
    tb_dataIn <= "11000101";
    wait;
end process;
```

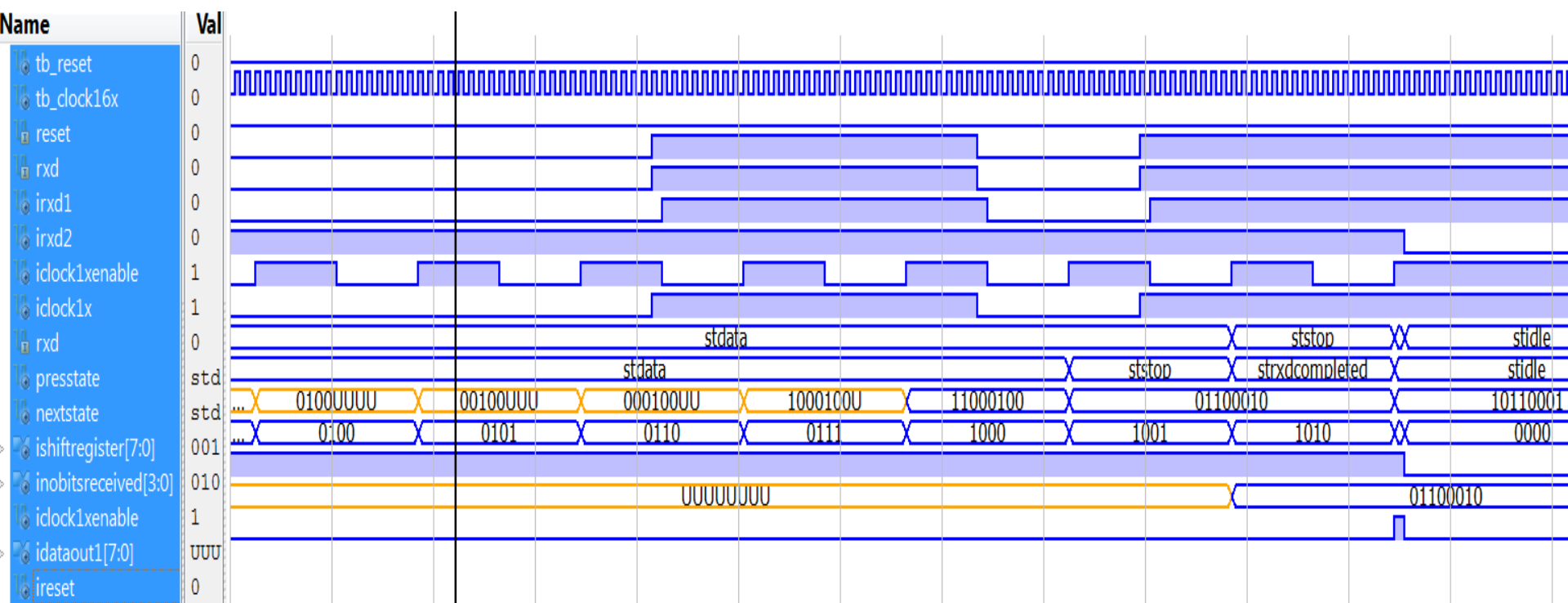
Rxd Validation- simulation

Rxd data is “01100010”



Rxd Validation- simulation

Rxd data is “01100010”



Summary

- VHDL Digital Design of serial communication bus RS232 & SPI (Specification, Design, Validation)
- VHDL Testbench for RS232
- VHDL constructs for Testbench Design
- Validation of Rx block of RS232 using simulation

Thank you

